

NETWORK SECURITY • INTRUSION DETECTION

NetSentinel

מערכת מבזרת לניטור תעבורת רשת ואיתור אנומליות

RSA

Npcap

SharpPcap

WPF

C# / .NET 8

נועם קמרי • בית ספר מקיף דרכא • 2025

מבוא ומוטיבציה

01

הבעיה

המשתמש הממוצע אינו יודע מה קורה ברשת שלו. פעילות חריגה עוברת ללא ידיעתו ולא ניתן לזהות אותה בזמן אמת.

02

הרעיון

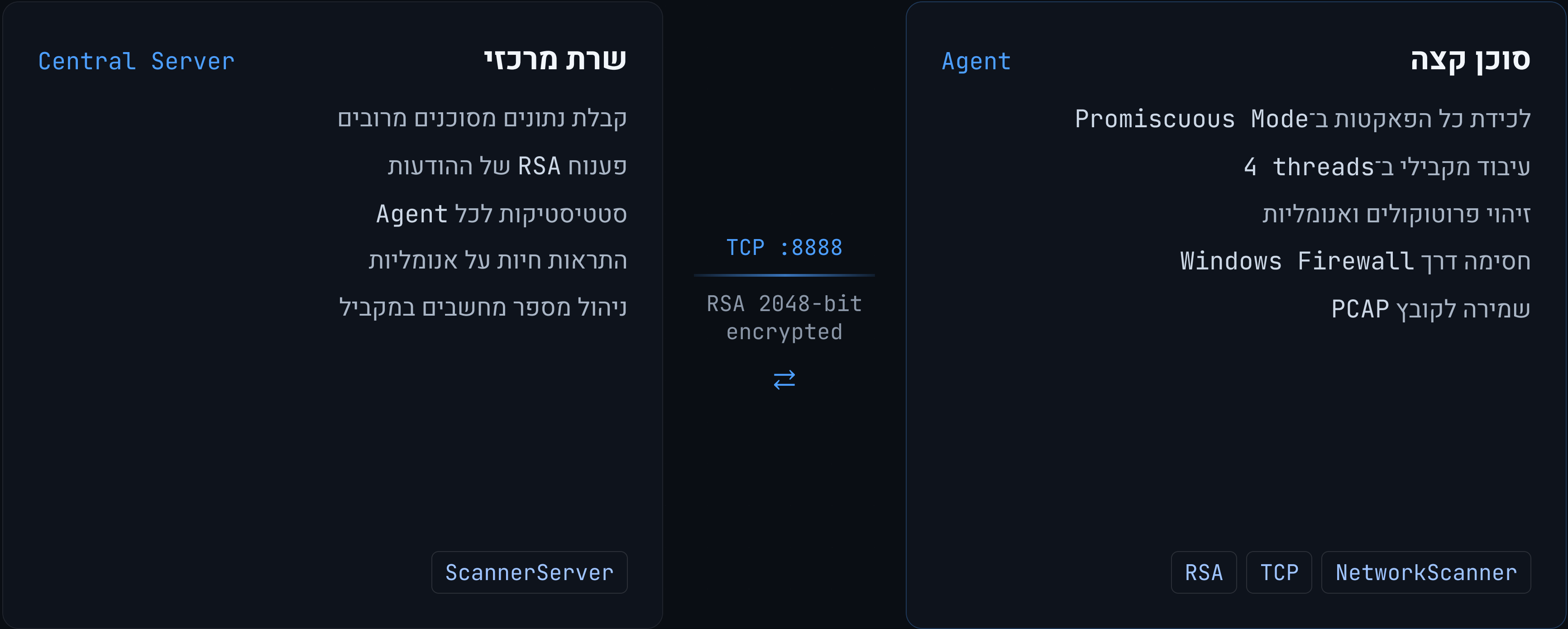
כלי פשוט ונגיש שנותן נראות מיידית לתנועה ברשת, מזהה אנומליות ומתריע עליהן אוטומטית.

03

המטרה

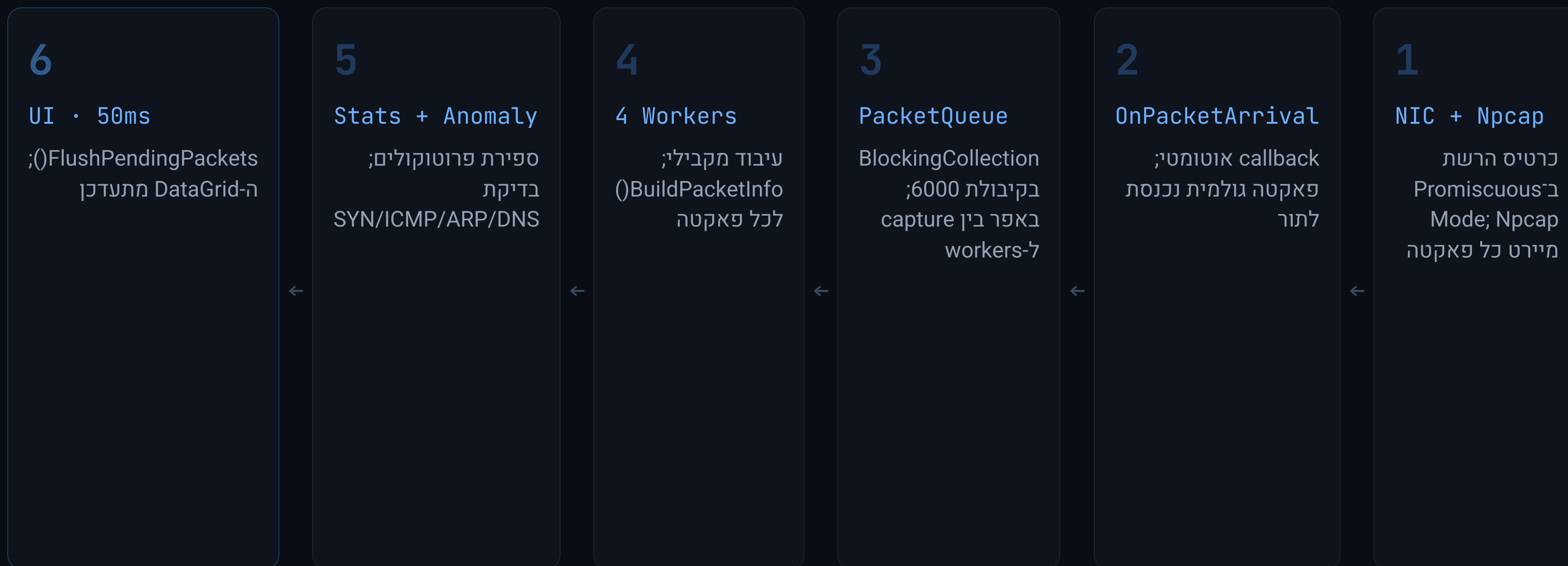
לכידת פאקטות, זיהוי אנומליות, חסימה אקטיבית וניהול מרכזי – הכל בכלי אחד קל להפעלה.

ארכיטקטורת המערכת



→ כל Agent מתחבר לשרת ב-TCP port 8888 ומצפין כל הודעה ב-RSA 2048-bit לפני שליחה.

זרימת נתונים – מהרשת עד ממשק המשתמש



! מקרה קצה: אם התור מלא – TryAdd() נכשל והפאקטה נפלטת, במקום לחסום את ה-capture thread.

Npcap & PacketQueue – לנידת פאקטות

PacketQueue



Promiscuous Mode

ברגיל ה-NIC מקבל רק פאקטות הנשלחות אליו
במצב זה ה-NIC מעביר **הכל** – גם פאקטות של מחשבים אחרים

דרישות הפעלה:

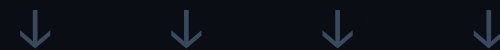
Administrator

Npcap installed

→ TryAdd אינו חוסם, ו-GetConsumingEnumerable() חוסם את ה-thread אוטומטית כשהתור ריק – ללא polling.

עיבוד מקבילי – 4 Worker Threads

PacketQueue · BlockingCollection



Thread 4

```
ParsePacket()  
BuildPacketInfo()  
UpdateStats()  
HandleArp()
```

Thread 3

```
ParsePacket()  
BuildPacketInfo()  
UpdateStats()  
HandleArp()
```

Thread 2

```
ParsePacket()  
BuildPacketInfo()  
UpdateStats()  
HandleArp()
```

Thread 1

```
ParsePacket()  
BuildPacketInfo()  
UpdateStats()  
HandleArp()
```

? למה 4? מספיק כדי לעבד burst של פאקטות במקביל בלי להציף את ה-CPU. BlockingCollection מסנכרן בין ה-threads ללא race conditions.

זיהוי פרוטוקולים – DetectProtocol()

שלב 1 – זיהוי שכבת התעבורה

UDP

Extract<UdpPacket>()

TCP

Extract<TcpPacket>()

ICMP

Extract<IcmpV4Packet>()

ARP

Extract<ArpPacket>()

שלב 2 – זיהוי פרוטוקול שכבת האפליקציה (TCP/UDP)

DHCP

Port 67 / 68 • UDP

DNS

Port 53 • UDP

HTTP

GET / POST / HEAD
/ HTTP

HTTPS

Payload[0]=0x16
Payload[1]=0x03 • TLS

→ עדיפות הזיהוי: בדיקת bytes ראשונה ← אחר כך port מוכר ← ב-default מוחזר 'UDP' / 'TCP'.

זיהוי אנומליות – SYN & ICMP Flood

איך עובד הזיהוי

ספי הזיהוי · Constants.cs

MAX_SYN_PER_SECOND

MAX_ICMP_PER_SECOND

למה thread נפרד?

אי אפשר לסנן flood per-packet – צריך לאסוף נתונים בחלון של שנייה. thread ייעודי עושה זאת בלי לעכב את עיבוד הפאקטות.

1 כל פאקטת TCP עם SYN=true, ACK=false נספרת per IP:Port

2 כל פאקטת ICMP נספרת per IP

3 thread נפרד מתעורר כל שנייה ובודק אם חרגנו מהסף

4 אם כן → RaiseAnomaly() → overlay אדום + שליחה לשרת + Block()

זיהוי אנומליות – Evil Twin · DNS Spoofing · ARP Spoofing

Evil Twin

איך מזהים

שומר BSSID+SSID בתחילת הריצה. כל 5 שניות בודק אם ה-BSSID השתנה וה-SSID זהה

למה מסוכן

רשת WiFi מזויפת עם אותו שם – BSSID שונה = נקודת גישה אחרת

DNS Spoofing

איך מזהים

כל DNS Response נרשם לפי Transaction ID. אותו ID מ-MAC שונה ← התראה (4 cooldown שניות)

למה מסוכן

תגובת DNS מזויפת מקדימה את השרת ומסיטה לאתר פיישינג

ARP Spoofing

איך מזהים

ConcurrentDictionary שומר MAC → IP שונה מהרשום ← התראה

למה מסוכן

תוקף מתחזה ל-IP אחר כדי להסיט אליו את התנועה

! כל אנומליה מציגה למשתמש overlay אדום עם הפרטים + כפתור חסימה מיידית + שליחה לשרת המרכזי.

חסימה אקטיבית – Windows Firewall

BlockIp(ip)

Evil Twin

ARP Spoofing

חוסם כל תנועה מה-IP

```
netsh advfirewall firewall
  add rule
  dir=in action=block
  protocol=any
  remoteip={ip}
```

BlockIcmp(ip)

ICMP Flood

חוסם כל ICMP מה-IP

```
netsh advfirewall firewall
  add rule
  dir=in action=block
  protocol=ICMP
  remoteip={ip}
```

Block(ip, port)

DNS Spoofing

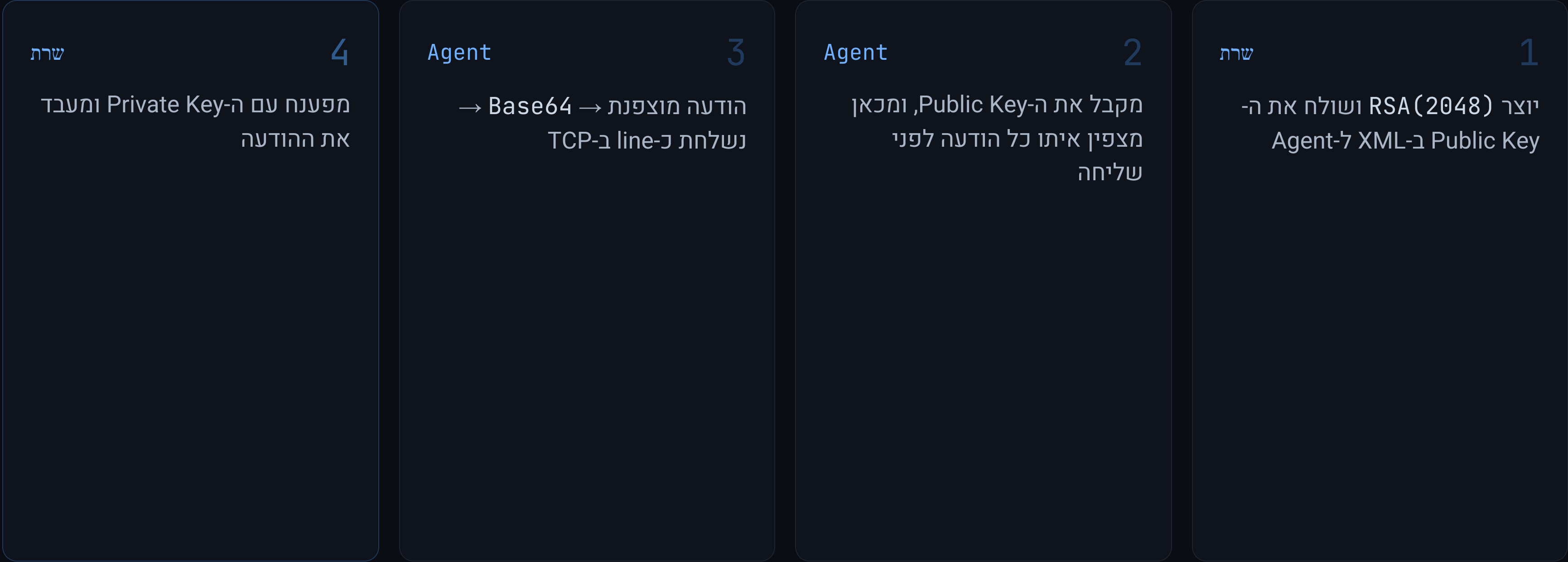
SYN Flood

חוסם TCP port ספציפי מה-IP

```
netsh advfirewall firewall
  add rule
  dir=in action=block
  protocol=TCP
  localport={port}
  remoteip={ip}
```

→ `ClearAllBlocks()` נקרא כשהמשתמש עוצר את הסריקה, ומוחק את כל חוקי ה-Firewall שנוצרו.

תקשורת מוצפנת – TCP + RSA



סוגי הודעות: HEARTBEAT STATS ALERT PROTOCOL_STATS

? למה Base64? כדי לשלוח bytes כטקסט רגיל על גבי חיבור ה-TCP.

שמירה וטעינה – PCAP File Format

מבנה קובץ PCAP

למה לממש ידנית?

הפרויקט משתמש ב־`PacketInfo` struct – פנימי ולא אובייקט של `SharpPcap`, ולכן אי אפשר להשתמש ב־`CaptureFileWriterDevice` ישירות.

הפתרון: `SaveToPcap()` ו־`LoadFromPcap()` ממומשים ידנית byte-by-byte לפי מפרט הפורמט.

✓ היתרון: תאימות מלאה עם Wireshark וכל כלי PCAP אחר. הטעינה מוולידצת את ה-Magic Number ומפענחת frame-by-frame.

Global Header

`Magic 0xa1b2c3d4 · Version · SnapLen`

24 B

Packet Header ×N

`Timestamp · Saved + Original Length`

16 B

Frame Data

`Ethernet(14) + IPv4(20) + ports(4) + padding`

data

סיכום – מה בנינו

01 · CAPTURE

לכידה

Npcap + Promiscuous Mode, תור
BlockingCollection

02 · PROCESS

עיבוד

Worker Threads 4 במקביל, זיהוי 8 פרוטוקולים

04 · BLOCK

חסימה

Windows Firewall דרך netsh, אוטומטית

05 · COMMS

תקשורת

TCP + RSA 2048-bit, שרת מרכזי מרובה סוכנים

03 · ANOMALY

אנומליות

SYN / ICMP Flood, ARP / DNS Spoofing, Evil
Twin

06 · PCAP

שמירה וטעינה

פורמט PCAP מלא, תאימות לכלי ניתוח